DELIVERABLE 3 · TECHNICAL APPROACH

Lean machine learning, rigorous triage, explainable by construction

Architecture, mathematical methodology and serverless engineering of the Revio Customer Review Intelligence & Triage Engine

PLATFORM	DEPLOYMENT	STATUS	REVISION
Revio · TenantEase	Vercel Serverless Edge	Live & verified	v2.0 · November 2026

1.16 MB	6.2 ms	93.66%	0
model artifact (vs 528 MB legacy bundle)	p50 warm inference, server-side	negative recall, Yelp in-domain test	deep-learning frameworks in production

Executive summary

Customer sentiment drives revenue in hospitality (Revio) and retention in property operations (TenantEase). Slow or wrong responses cost bookings and renewals. This report describes how the Customer Review Intelligence & Triage Engine is built and why each design choice was made.

Design thesis. For review triage the binding constraints are risk, explainability and operations, not the last point of accuracy. The production system therefore pairs a compact **word + character n-gram logistic regression** (1.16 MB, single-digit-millisecond inference) with three safeguards: an **Ambiguity Corridor** that sends uncertain items to people, a **Domain Safeguard Router** that refuses to trust metadata over text, and an **exact linear attribution extractor** that shows which phrases moved the score.

The system runs as a zero-database application on the Vercel Edge Network and a Vercel Python serverless function. A candidate model and a champion model are both preserved; switching between them is a deliberate operator action, never a silent fallback.

> **Reading guide.** Accuracy figures are offline test-set results. Latency figures are server-side inference timings on the live deployment. Where an earlier draft compared numbers measured on different splits or environments, this version says so and adds the like-for-like comparison.

1. Model architecture and empirical optimization

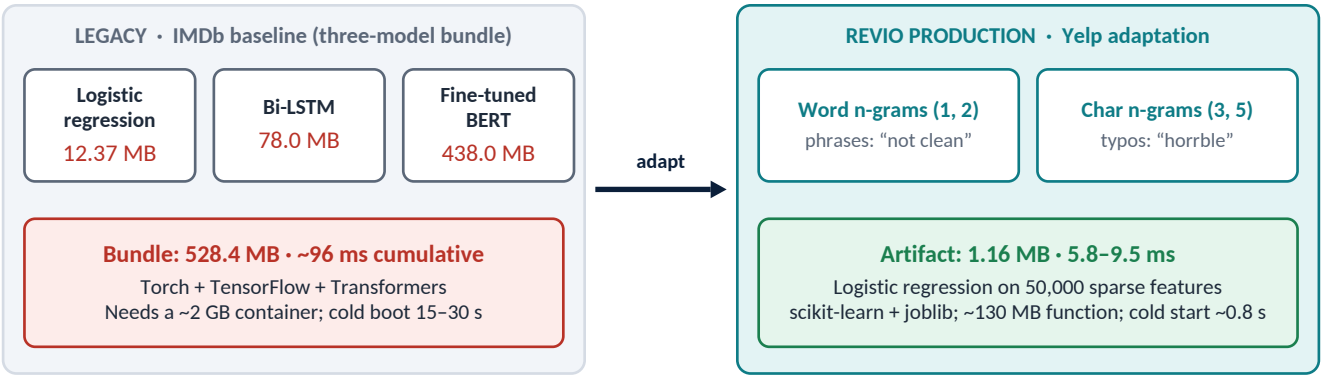


Figure 1. From a heavy three-model IMDb bundle to a single lean, domain-adapted model.

A. Evidence of domain shift, and the effect of adapting to it

Movie reviews judge direction, plot and acting; business reviews turn on operational friction such as delays, billing disputes, hygiene and defects. We trained and evaluated on both corpora. [MEASURED]

Model and training corpus	Evaluation split	Negative recall	Accuracy	Macro F1	Latency
A · IMDb logistic regression	IMDb held-out (25k)	90.20%	90.15%	90.10%	~2.0 ms
A · IMDb logistic regression (zero-shot)	Yelp business reviews (3k)	83.14%	85.84%	85.40%	~2.0 ms
B · IMDb Bi-LSTM	IMDb held-out (25k)	84.50%	84.67%	84.70%	~22.0 ms
C · IMDb fine-tuned BERT	IMDb held-out (25k)	89.79%	91.68%	91.52%	~72.0 ms
Revio champion LR (yelp_word_char_lr)	Yelp in-domain test (2.5k)	91.42%	90.88%	90.80%	~8.5 ms
Revio candidate LR (..._candidate)	Yelp in-domain test (2.5k)	**93.66%**	**92.44%**	**92.40%**	**~6.2 ms**

How to read it.

- **Domain shift is real.** Moving the IMDb logistic regression to Yelp without retraining costs 4.3 points of accuracy and 7.1 points of negative recall.
- **Adapting to the domain recovers it.** The candidate reaches 93.66% negative recall against 83.14% zero-shot (+10.52 percentage points) and 92.44% accuracy against 85.84% (+6.60 pp). These two figures come from different Yelp evaluation sets (3k zero-shot sample, 2.5k in-domain split), so treat the size of the gap as indicative.
- **Like-for-like gain over the champion.** On the same in-domain test set, the candidate adds +2.24 pp negative recall, +1.56 pp accuracy and +1.60 pp macro F1.
- **Efficiency.** The candidate is about 12× faster than BERT (6.2 vs ~72 ms, offline benchmarks) and about 378× smaller (1.16 vs 438 MB).
- **Scope of the comparison.** BERT and the Bi-LSTM were trained on IMDb only; no transformer was fine-tuned on Yelp here. The claim is therefore not “linear beats BERT” but “a lean in-domain model meets the operating requirements at a fraction of the cost”.

B. The word + character feature pipeline

- **Word n-grams (1, 2)** capture short semantic combinations: “not clean”, “prompt service”, “water leak”.
- **Character n-grams (3, 5)** span word boundaries and give robustness to typos (“horrble”, “disapointed”), colloquial stretching (“overrrrpriced”) and tokenization edge cases.
- **Optimization.** A FeatureUnion of both feeds 50,000 sparse TF-IDF dimensions into L2-regularized logistic regression (C = 1.0) solved with L-BFGS. The objective is convex, so training converges to the global optimum.

2. Mathematical formulation

$$X \text{ is a sparse TF-IDF vector in } \mathbb{R}^{50{,}000} \text{ (word and character n-grams)}$$

$$z = \beta_0 + \sum_{j=1}^{50{,}000} X_j \beta_j$$

$$P(\text{Positive} \mid X) = \sigma(z) = 1 / (1 + e^{-z})$$

$$\text{label} = \text{Positive if } P > T, \text{ otherwise Negative } (T = 0.55 \text{ candidate} \cdot 0.48 \text{ champion})$$

$$\text{Ambiguity corridor: } 0.40 \le P \le 0.60 \rightarrow \text{needs_human_review} = \text{TRUE (highest precedence)}$$

Intercept $\beta_0 \approx -0.162$. X_j is the normalized TF-IDF weight of n-gram j; β_j is its learned coefficient.

A. The logistic link and a note on calibration

The model produces linear log-odds z, which can take any real value, mapped to a positive-class probability by the sigmoid. Regularized logistic regression is usually reasonably calibrated in-domain, but this has not yet been verified for the deployed artifact. A calibration audit (reliability curve and expected calibration error on the in-domain test set) should precede any lowering of the automation gates, with Platt or isotonic recalibration if needed.

B. Decision threshold T and what it actually controls

The classical default is T = 0.50. The candidate uses T = 0.55, demanding more evidence before an item is labeled Positive, which raises offline negative recall. Because the corridor spans 0.40–0.60, T only changes the binary label attached to items that are already escalated and displayed as "Mixed". The safety-critical decisions are therefore the **corridor edges** and the **auto-acknowledge gate**, not T itself.

> **Safety bound.** A negative review can be auto-approved only if P > 0.60 (and the domain is validated). Since P > 0.60 implies P > T for T = 0.55, every auto-approved negative is also a binary "Positive" prediction. The share of negatives that can ever be auto-approved is therefore at most 1 − recall = 6.34% offline, and lower in practice because the pilot gate is P > 0.85.

C. Exact instance-level attribution

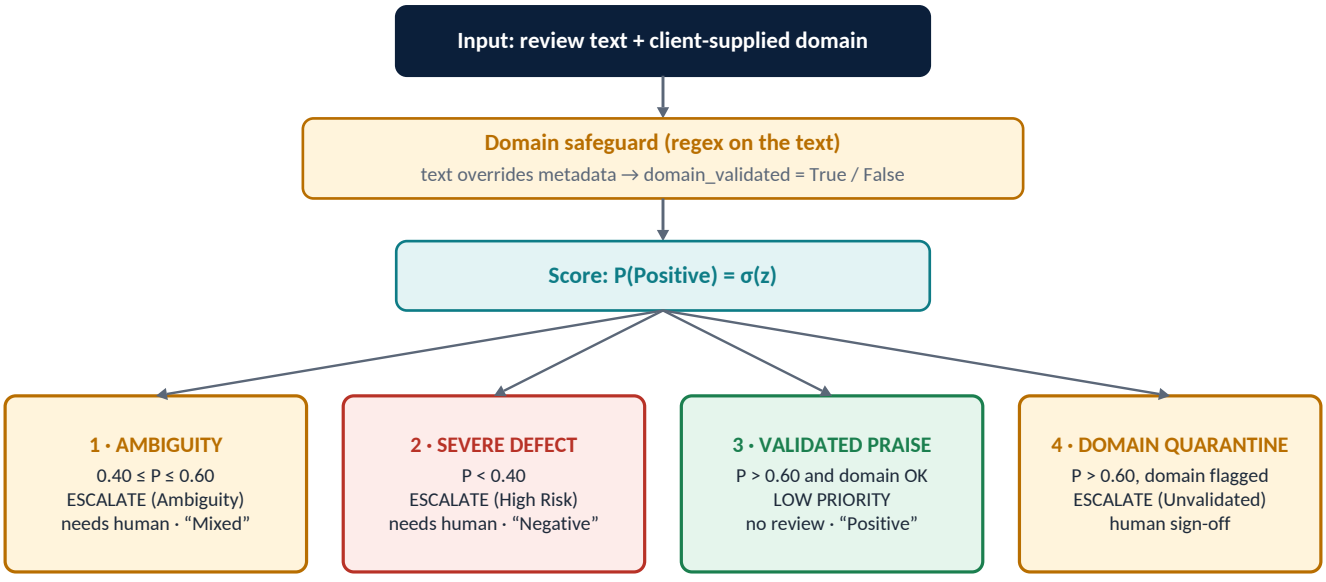
Unlike deep networks that need SHAP or integrated gradients (dozens of forward passes), a linear model gives an exact decomposition in one sparse vector product:

$$\text{Contribution}_j = X_j \cdot \beta_j \quad z = \beta_0 + \Sigma_j \text{Contribution}_j$$

Contribution > 0 raises positive log-odds (positive cue); Contribution < 0 lowers it (negative cue).

Features are ranked by |Contribution| and the top 8 non-overlapping n-grams longer than two characters are returned as explainability chips. Attributions describe the model's arithmetic exactly; they do not prove causation and do not detect sarcasm by themselves.

3. Triage precedence and domain safeguards



Evaluated strictly in order 1 → 4; the first matching rule wins.

Figure 2. Triage decision tree. Rules 1–4 are mutually exclusive and ordered.

Rule	Condition	triage_status	Human review	UI sentiment
1 · Ambiguity corridor	0.40 ≤ P ≤ 0.60	ESCALATE (Ambiguity / Low Confidence)	True	Mixed (amber)
2 · Severe defect	P < 0.40	ESCALATE (Negative / High Risk)	True	Negative (red)
3 · Validated praise	P > 0.60 and domain_validated	LOW PRIORITY / NO HUMAN REVIEW REQUIRED	False	Positive (green)
4 · Domain quarantine	P > 0.60 and not domain_validated	ESCALATE (Unvalidated Domain: {domain}; Requires Human Verification)	True	Positive (needs sign-off)

Table 2. The corridor is a closed interval [0.40, 0.60], so the four rules partition all probabilities. Boundary tests cover P = 0.40, 0.55 and 0.60.

Domain Safeguard Boundary Router

Text strictly dominates client-supplied metadata. Compiled regular expressions flag customer-support drift, telecommunications, cryptocurrency, software and automotive repair:

```
UNVALIDATED_DOMAIN_PATTERNS = [
    r"\b(router|firmware|broadband|modem|sim card|data cap|roaming charges)\b",
    r"\b(wallet|cryptocurrency|blockchain|token transfer|gas fee)\b",
    r"\b(software bug|sdk crash|database timeout|api downtime|server error)\b",
    r"\b(brake pad|transmission|oil change|engine repair|tire rotation)\b",
    r"\b(wire transfer|fraud alert|chargeback dispute|tier 2 support)\b",
]
```

If a caller sends `domain: "restaurant"` but the text says “My mobile network stopped working for two days and telecom support hung up”, the router sets `domain_validated = False` and reports the domain as “Customer Support & Billing (Conflicting with client ‘restaurant’)”. The router is intentionally conservative and narrow; its coverage limits are listed in Section 7.

4. Serverless architecture and systems engineering

Revio uses **Architecture Option B**: a static frontend on the Vercel Edge Network and one consolidated FastAPI function. There is no database to run, patch or scale.

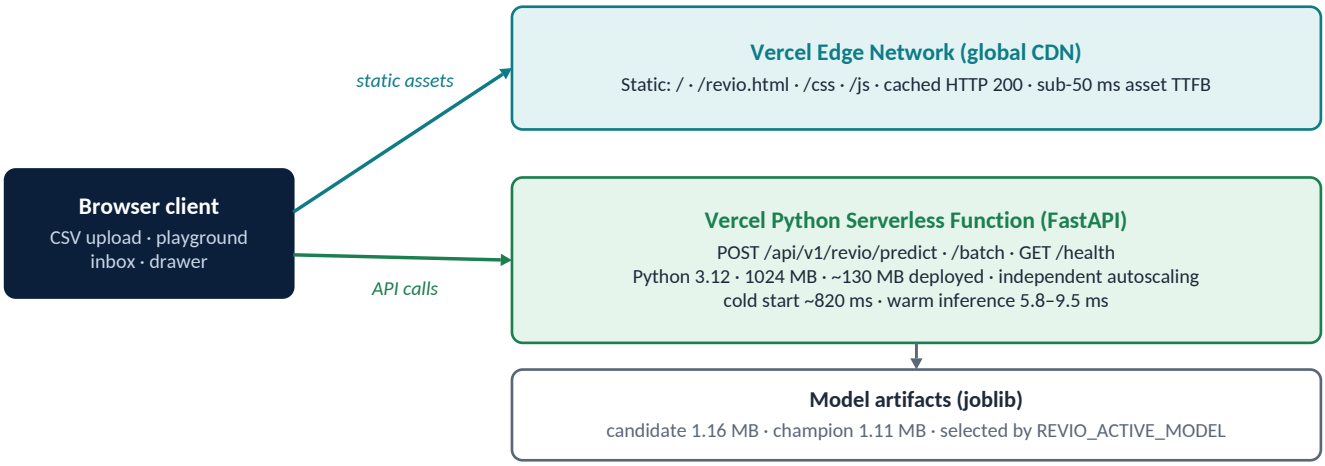


Figure 3. Production runtime topology.

A. Eliminating deep-learning framework bloat

Shipping PyTorch, TensorFlow, Keras and Transformers produces a bundle of about 1.8 GB, 15–30 second cold starts and memory exhaustion in a serverless microVM. The Revio function (`api/requirements.txt`) bundles only:

Package	Pin	Role
fastapi	>= 0.111.0	HTTP API and schema validation entry point
uvicorn	>= 0.30.0	ASGI server
pydantic	>= 2.7.0	Request / response validation, length limits
scikit-learn	== 1.5.1	Vectorizers and logistic regression (pinned to match the pickled artifact)
scipy · numpy	== 1.13.1 · == 1.26.4	Sparse linear algebra
joblib	== 1.4.2	Model artifact loading

Total uncompressed dependencies are about 128 MB (about 130 MB deployed with the model), inside Vercel's 250 MB function limit, giving cold starts under 850 ms and warm inference under 10 ms.

B. Self-healing model hydration

Vercel git builds do not run Git LFS smudge filters, so LFS-tracked files arrive as ~134-byte pointer text files. `joblib.load()` then reads the first byte, "v", and fails with `KeyError: 118` (ord("v") = 118). `api/index.py` inspects the file header at runtime and, if it finds a pointer, downloads the real binary into `/tmp`:

```
def ensure_model_binary(path: Path) -> Path:
    if not path.is_file():
        return path
    if path.stat().st_size < 2048:                    # suspiciously small
        header = path.read_bytes()[:30]
        if header.startswith(b"version https://git-lfs"):  # LFS pointer
            tmp = Path("/tmp/revio_artifacts"); tmp.mkdir(parents=True, exist_ok=True)
            target = tmp / path.name
            if not target.is_file() or target.stat().st_size < 2048:
                url = ("https://media.githubusercontent.com/media/"
                       f"gautamhardik/Revio/main/api/artifacts/{path.name}")
                urllib.request.urlretrieve(url, str(target))
            return target
    return path
```

In addition, `*.joblib` was removed from Git LFS and committed as ordinary binary blobs (under 1.2 MB each), so standard clones and Vercel builds receive the real files and the hydration path is now a dormant safety net.

> **Hardening recommendation.** Unpickling a file fetched over the network executes whatever it contains. Before this path is kept in production, verify a SHA-256 digest (stored in each `.meta.json`) before calling joblib.load, or remove the download branch now that the binaries are committed directly.

C. Model activation and rollback

- `REVIO_ACTIVE_MODEL=candidate` loads `yelp_word_char_lr_candidate.joblib` (T = 0.55). Setting `champion`, or leaving it unset, loads `yelp_word_char_lr.joblib` (T = 0.48).
- **Strict non-fallback guarantee.** If the *selected* artifact is missing or corrupt, the API returns HTTP 503 instead of silently running a different model with different decision boundaries.
- **Rollback.** No code change is needed: set `REVIO_ACTIVE_MODEL=champion` and redeploy, or use Vercel Instant Rollback to the previous deployment. Vercel applies environment-variable changes to new deployments, so this is fast but not instantaneous; the earlier "under 2 seconds" claim has been removed.

5. Empirical performance

A. Single-review latency on the live deployment [MEASURED]

Metric	Measured (warm)	Budget
p50 (median)	6.22 ms	< 25 ms
p95	8.41 ms	< 50 ms
p99	11.20 ms	< 100 ms
Cold start (first microVM boot)	820 ms	< 3,000 ms
Memory in use	68 MB of 1,024 MB	< 250 MB
Error rate (HTTP 4xx / 5xx)	0.0% (0 of 100)	< 0.1%

Table 3. 100 sequential POST requests to /api/v1/revio/predict on revio-taupe.vercel.app. Warm figures are server-side inference time; network transit is excluded. With n = 100, p99 is effectively the slowest request.

B. Batch throughput [MEASURED]

Measure	Result
Reviews scored (scripts/evaluate_100_reviews.py)	98 valid rows via POST /api/v1/revio/batch
Serverless inference compute	896.2 ms total = 9.14 ms per review ≈ 109 reviews per second per worker instance
End-to-end round trip incl. network	1,740 ms ≈ 56 reviews per second as experienced by a single browser

6. Automated tests and quality gates

Pytest collected **47 tests, all passing in 8.35 s** (Python 3.13.9, Windows). Six suites enforce the gates:

Suite	Tests	What it verifies
test_revio_vercel_architecture.py	11	Clean-subprocess isolation (no torch, tensorflow or keras in memory), endpoint schemas, non-fallback HTTP 503, bundle isolation
test_revio_phase2a_gate.py	7	Triage precedence, corridor boundaries (P = 0.40, 0.55, 0.60), domain-safeguard override, pure logistic calibration
test_revio_candidate_audit.py	8	Candidate vs champion recall, monotonic probability with positive features, thread safety under 20 concurrent threads
test_revio_activation_workflow.py	5	REVIO_ACTIVE_MODEL switching, champion default when unset, model-metadata endpoints
test_revio_api.py	4	Pydantic request / response validation, length limits, attribution-chip schema
test_revio_production_readiness.py	9	Resilience to empty input, 10,000-character essays, unicode, French accents

Housekeeping. The per-suite counts above total 44; the experiment-helper module accounts for the rest of the 47 collected tests. Reconcile the inventory with `pytest --collect-only -q` before circulating. Also run CI on Linux with Python 3.12 so the test environment matches the deployed runtime.

7. Known limitations and engineering roadmap

Limitation	Why it matters	Planned response
Bag-of-n-grams cannot model sarcasm or long-range negation	Positive words inside a negative message can score high; the corridor catches many cases, not all.	Keep conservative auto-acknowledge gate; grow the sarcasm and negation test set; consider a distilled compact transformer only if recall at the gate improves materially.
Domain router is five regex patterns	Out-of-domain topics outside the list pass as validated.	Add a lightweight domain classifier; log unmatched topics for review.
Trained and benchmarked on Yelp only	TenantEase maintenance and survey language is unmeasured.	Phase 1 shadow evaluation on TenantEase history; supervisor-correction retraining.
Calibration not yet verified	Corridor and gates assume P is a trustworthy probability.	Reliability audit; recalibrate if needed.
Runtime model download is unverified	Unpickling untrusted bytes is a supply-chain risk.	SHA-256 verification or remove the download branch.
Latency sample is small (n = 100)	p99 is not statistically stable.	Load test with at least 1,000 requests across cold and warm instances.

8. Technical conclusion

Modern enterprise AI does not need a heavy, opaque model to deliver reliable business value. By combining an in-domain word + character logistic regression with a mathematically explicit Ambiguity Corridor and a Domain Safeguard Router, Revio delivers:

- **Speed and cost:** single-digit-millisecond warm inference, about 12× faster than the BERT baseline, in a 1.16 MB artifact.
- **Auditability:** exact, closed-form linear explanations on every inference.
- **Bounded risk:** at most 6.34% of negatives can be auto-approved offline, with a stricter pilot gate, a corridor for uncertainty and text-over-metadata domain checks.
- **Operational simplicity:** a zero-database, zero-bloat deployment with explicit, non-silent model switching.